

引论

1. 编译器的作用: 将高级编程语言编写的源程序转换成目标机器可执行的机器代码。主要编译阶段: 词法分析 → 语法分析 → 语义分析 → 中间代码生成 → 代码优化 → 目标代码生成

2. 前端 (与源语言相关): 词法分析、语法分析、语义分析、中间代码生成

后端 (与目标机器相关): 代码优化、目标代码生成、存储分配、寄存器分配

3. 作用: 根据语言的语法规则, 将词法单元序列组织成语法树。

常见分析方法: 自顶向下分析: LL(1)分析法、递归下降分析、预测分析

自底向上分析: LR(0)分析法、SLR(1)分析法、LR(1)分析法、LALR(1)分析法

4. 作用: 将源程序文本分解成词素序列、过滤注释和空白字符、记录源程序位置信息

输入: 源程序字符串

输出: 词素(Token)序列, 每个词素包含: 类型 (如标识符、关键字、运算符等)、属性值 (如标识符的名字、常量的值等)

5. 作用: 提供源程序的中间表示形式、便于代码优化、提高编译器的可移植性

常见形式: 三地址码、四元式、抽象语法树(AST)、静态单赋值形式(SSA)、P-代码

词法分析

1. (00111101110)*(00111101110)?

2. NFA 图: 设 q0: 初始状态 q1: 接受第一个字符(a or b) q2: 接受有限个 aa 或 bb 后 q3: 接受状态

状态转移图为:

(q0)--a/b-->(q1);(q1)--aa/bb-->(q2);(q2)--aa/bb-->(q2);(q2)--a/b-->(q3)

根据 DFA 图的定义, 等价变换后的 DFA 图与此 NFA 图相同且已为最简形式。

编程如下:

```
#include <stdio.h>
#include <string.h>
#include <stdbool.h>
// 检查字符串是否匹配模式
bool matches_pattern(const char* str) {
    int len = strlen(str);
    if (len < 3)
        return false; // 最短的合法字符串长度为 3 // 检查第一个字符
```

```

if (str[0] != 'a' && str[0] != 'b') return false;
// 检查中间部分
int i = 1;
while (i < len - 1) {
    if (i + 1 < len && ((str[i] == 'a' && str[i+1] == 'a') || (str[i] == 'b' && str[i+1]
== 'b')))) { i += 2; }
    else { break;
    }
}
// 检查最后一个字符
if (i == len - 1 && (str[i] == 'a' || str[i] == 'b')) {
    return true; }
return false; }
// 在字符串中查找所有匹配的子串
void find_matches(const char* str) {
    int len = strlen(str);
    char* substr = (char*)malloc((len + 1) * sizeof(char));
    // 尝试所有可能的子串
    for (int i = 0; i < len; i++) {
        for (int j = 3; i + j <= len; j++) {
            // 最短长度为 3
            strncpy(substr, str + i, j); substr[j] = '\0';
            if (matches_pattern(substr)) {
                printf("匹配: %s\n", substr);
            }
        }
    } free(substr);
}
int main() {
    // 测试用例
    const char* test_strings[] = { "aaa", // 匹配 "aba", // 匹配 "aabbaa", // 匹配
"abbbba", // 匹配 "aa", // 不匹配 "abba", // 不匹配 };
    int num_tests = sizeof(test_strings) / sizeof(test_strings[0]);
    for (int i = 0; i < num_tests; i++) {
        printf("\n 测试字符串: %s\n", test_strings[i]); find_matches(test_strings[i]); }
    return 0;
}

```

3.步骤为：1)为 DFA 添加新的开始状态和接受状态。2)对于每个中间状态，使用以下公式消除它： $R = R1(R2)^*R3$ ，其中，R1 是从前一状态到当前状态的转换，R2 是当前状态到自身的转换，R3 是从当前状态到下一状态的转换。3)重复步骤 2 直到只剩下开始状态和接受状态 4)最终路径上的表达式就是等价的正则表达式

4.使用 Myhill-Nerode 定理，检查是否存在等价状态。具体步骤为：

1)创建状态对的区分表

- 2)初始标记所有终态和非终态的状态对
- 3)递归标记那些经过相同输入转到已标记状态对的状态对
- 4)如果还存在未标记的状态对，说明 DFA 不是最小的
- 5)如果所有状态对都被标记，则 DFA 已经是最小的

语法分析

1.语法分析过程如下:

步骤	状态栈	符号栈	输入串	动作
0	0	\$	aaabbb\$	移入 a
1	0 2	\$a	aabbb\$	移入 a
2	0 2 2	\$aa	abbb\$	移入 a
3	0 2 2 2	\$aaa	bbb\$	归约 $S \rightarrow \epsilon$
4	0 2 2 2 1	\$aaaS	bbb\$	移入 b
5	0 2 2 2 1 3	\$aaaSb	bb\$	归约 $S \rightarrow aSb$
6	0 2 2 1	\$aaS	bb\$	移入 b
7	0 2 2 1 3	\$aaSb	b\$	归约 $S \rightarrow aSb$
8	0 2 1	\$aS	b\$	移入 b
9	0 2 1 3	\$aSb	\$	归约 $S \rightarrow aSb$
10	0 1	\$S	\$	接受

2.语法分析过程如下:

步骤	分析栈	输入串	推导所用产生式
1	\$E	id+idid\$	$E \rightarrow E+E$
2	\$+EE	id+idid\$	$E \rightarrow id$
3	\$+Eid	+idid\$	匹配 id
4	\$+E	+idid\$	匹配 +
5	\$E	idid\$	$E \rightarrow EE$
6	\$EE	idid\$	$E \rightarrow id$
7	\$Eid	id\$	匹配 id

8	\$E	id\$	$E \rightarrow id$
9	\$id	id\$	匹配 id
10	\$	\$	接受

3.语法分析过程如下:

步骤	状态栈	符号栈	输入串	动作
0	0	\$	acccdbd\$	移入 a
1	0 2	\$a	cccd bd\$	移入 c
2	0 2 4	\$ac	ccdbd\$	移入 c
3	0 2 4 4	\$acc	cdbd\$	移入 c
4	0 2 4 4 4	\$accc	dbd\$	归约 $T \rightarrow \epsilon$
5	0 2 4 4 4 5	\$acccT	dbd\$	移入 d
6	0 2 4 4 4 5 6	\$acccTd	bd\$	归约 $T \rightarrow cTd$
7	0 2 4 4 5	\$accT	bd\$	移入 b
8	0 2 4 4 5 3	\$accTb	d\$	移入 d
9	0 2 4 4 5 3 6	\$accTbd	\$	归约 $S \rightarrow aSb$
10	0 1	\$S	\$	接受

4.语法分析过程如下:

步骤	状态	符号栈	输入串	动作
0	0	\$	id*(id+id)\$	移入 id
1	0 5	\$id	*(id+id)\$	归约 $F \rightarrow id$
2	0 3	\$F	*(id+id)\$	归约 $T \rightarrow F$
3	0 2	\$T	*(id+id)\$	移入 *

4	0 2 7	\$T*	(id+id)\$	移入(
5	0 2 7 4	\$T*(	id+id)\$	移入 id
6	0 2 7 4 5	\$T*(id	+id)\$	归约 $F \rightarrow id$
7	0 2 7 4 3	\$T*(F	+id)\$	归约 $T \rightarrow F$
8	0 2 7 4 2	\$T*(T	+id)\$	移入 +
9	0 2 7 4 2 6	\$T*(T+	id)\$	移入 id
10	0 2 7 4 2 6 5	\$T*(T+id	)\$	归约 $F \rightarrow id$
11	0 2 7 4 2 6 3	\$T*(T+F	)\$	归约 $T \rightarrow F$
12	0 2 7 4 2 6 2	\$T*(T+T	)\$	归约 $E \rightarrow T$
13	0 2 7 4 2	\$T*(E	)\$	移入)
14	0 2 7 4 2 8	\$T*(E)	\$	归约 $F \rightarrow (E)$
15	0 2 7 3	\$T*F	\$	归约 $T \rightarrow T*F$
16	0 2	\$T	\$	归约 $E \rightarrow T$
17	0 1	\$E	\$	接受

5.语法分析过程如下:

步骤	状态	符号栈	输入串	动作
0	0	\$	ababcc\$	移入 a
1	0 2	\$a	babcc\$	移入 b
2	0 2 3	\$ab	abcc\$	移入 a
3	0 2 3 2	\$aba	bcc\$	移入 b

4	0 2 3 2 3	\$abab	cc\$	移入 c
5	0 2 3 2 3 4	\$ababc	c\$	归约 $S \rightarrow c$
6	0 2 3 2 3 1	\$ababS	c\$	移入 c
7	0 2 3 2 3 1 4	\$ababSc	\$	归约 $S \rightarrow c$
8	0 2 3 2 3 1	\$ababS	\$	归约 $S \rightarrow bS$
9	0 2 3 2 1	\$abaS	\$	归约 $S \rightarrow aS$
10	0 2 3 1	\$abS	\$	归约 $S \rightarrow bS$
11	0 2 1	\$aS	\$	归约 $S \rightarrow aS$
12	0 1	\$S	\$	接受

6.语法分析过程如下:

步骤	状态	符号栈	输入串	动作
0	0	\$S	acbcc\$	$S \rightarrow aSb$
1	0 1	\$aSb	acbcc\$	移入 a
2	0 1 2	\$aSb	cbcc\$	$S \rightarrow cSc$
3	0 1 2 3	\$acSc	bcc\$	$S \rightarrow \epsilon$
4	0 1 2 3 4	\$acSc	bcc\$	移入 c
5	0 1 2 3 4 5	\$acc	cc\$	移入 c
6	0 1 2 3 4 5 6	\$accc	\$	归约 $S \rightarrow cSc$
7	0 1	\$aS	\$	移入 b
8	0 1 7	\$aSb	\$	归约 $S \rightarrow aSb$

9	0	\$S	\$	接受
---	---	-----	----	----

7.语法分析过程如下:

步骤	状态	符号栈	输入串	动作
0	0	\$	aabcc\$	移入 a
1	0 2	\$a	abcc\$	移入 a
2	0 2 2	\$aa	bcc\$	移入 b
3	0 2 2 3	\$aab	cc\$	移入 c
4	0 2 2 3 4	\$aabc	c\$	移入 c
5	0 2 2 3 4 4	\$aabcc	\$	归约 $S \rightarrow cc$
6	0 2 2 3 1	\$aabS	\$	归约 $S \rightarrow bS$
7	0 2 2 1	\$aaS	\$	归约 $S \rightarrow aS$
8	0 2 1	\$aS	\$	归约 $S \rightarrow aS$
9	0 1	\$S	\$	接受

8.语法分析过程如下:

步骤	状态	符号栈	输入串	动作
0	0	\$	abbccc\$	移入 a
1	0 2	\$a	bbccc\$	移入 b
2	0 2 3	\$ab	bccc\$	移入 b
3	0 2 3 3	\$abb	ccc\$	移入 c
4	0 2 3 3 4	\$abbc	cc\$	移入 c

5	0 2 3 3 4 4	\$abbcc	c\$	移入 c
6	0 2 3 3 4 4 4	\$abcccc	\$	归约 $S \rightarrow ccc$
7	0 2 3 3 1	\$abbS	\$	归约 $S \rightarrow bS$
8	0 2 3 1	\$abS	\$	归约 $S \rightarrow bS$
9	0 2 1	\$aS	\$	归约 $S \rightarrow aS$
10	0 1	\$S	\$	接受

9.语法分析过程如下:

步骤	状态	符号栈	输入串	动作
0	0	\$	abcccc\$	移入 a
1	0 2	\$a	bcccc\$	移入 b
2	0 2 3	\$ab	cccc\$	移入 c
3	0 2 3 4	\$abc	ccc\$	移入 c
4	0 2 3 4 4	\$abcc	cc\$	移入 c
5	0 2 3 4 4 4	\$abccc	c\$	移入 c
6	0 2 3 4 4 4 4	\$abcccc	\$	归约 $S \rightarrow cccc$
7	0 2 3 1	\$abS	\$	归约 $S \rightarrow bS$
8	0 2 1	\$aS	\$	归约 $S \rightarrow aS$
9	0 1	\$S	\$	接受

10.语法分析过程如下:

0	0	\$	abcccc\$	移入 a
---	---	----	----------	------

1	0 2	\$a	bcccccc\$	移入 b
2	0 2 3	\$ab	cccccc\$	移入 c
3	0 2 3 4	\$abc	cccc\$	移入 c
4	0 2 3 4 4	\$abcc	ccc\$	移入 c
5	0 2 3 4 4 4	\$abccc	cc\$	移入 c
6	0 2 3 4 4 4 4	\$abcccc	c\$	移入 c
7	0 2 3 4 4 4 4 4	\$abccccc	\$	归约 $S \rightarrow ccccc$
8	0 2 3 1	\$abS	\$	归约 $S \rightarrow bS$
9	0 2 1	\$aS	\$	归约 $S \rightarrow aS$
10	0 1	\$S	\$	接受

语义分析

1.	产生式规则	语义规则
	$E \rightarrow E1 + T$	$E.val = E1.val + T.val$
	$E \rightarrow T$	$E.val = T.val$
	$T \rightarrow T1 * F$	$T.val = T1.val * F.val$
	$T \rightarrow F$	$T.val = F.val$
	$F \rightarrow integer$	$F.val = integer.lexval$

编程实现:

```
// 节点结构
struct Node {
    int val; // 存储计算结果
    char type; // 节点类型 };
// 计算表达式值的函数示例
int evaluate(Node* root) {
    if (!root) return 0;
    switch(root->type) {
        case INTEGER:
            return root->val;
        case PLUS:
```

```

        return evaluate(root->left) + evaluate(root->right);
    case MULTIPLY:
        return evaluate(root->left) * evaluate(root->right); }
    return 0;
}

```

2.

产生式规则

$\text{stmt} \rightarrow \text{id1} = \text{E}$

$\text{E} \rightarrow \text{id}$

$\text{E} \rightarrow \text{int}$

语义规则

if $\text{id1.type} \neq \text{E.type}$ then
 error("类型不匹配")

endif

$\text{E.type} = \text{lookup}(\text{id.name})$ // 查找
 符号表获取类型

$\text{E.type} = \text{integer}$

编程实现:

// 类型检查函数示例

```
bool typeCheck(Node* stmt) {
```

```
    if (!stmt) return true;
```

```
    if (stmt->type == ASSIGNMENT) {
```

```
        int leftType = getVariableType(stmt->left->name); // 从符号表获取左侧  
        变量类型
```

```
        int rightType = getExpressionType(stmt->right); // 计算右侧表达式类型
```

```
        if (leftType != rightType) {
```

```
            printf("类型错误: 变量 %s 的类型与表达式类型不匹配
```

```
            \n", stmt->left->name);
```

```
            return false; }
```

```
        }
```

```
    return true;
```

```
}
```